

Retrieval-Augmented Chatbots (RAG) and Their Implementation

Module 1: Introduction

What is a Chatbot?

A chatbot is a software system capable of interacting with humans through natural language.

Examples include:

- Customer support bots
- Virtual assistants
- Educational tutors
- Medical assistants
- Legal assistants

Traditional chatbots relied on:

- Rule-based systems
- Decision trees
- Intent classification
- Predefined responses

Modern chatbots use **Large Language Models (LLMs)** such as:

- ChatGPT
 - Claude
 - Gemini
 - Llama
-

Why Do LLM Chatbots Need Retrieval?

Although LLMs are powerful, they have several limitations.

1. Hallucination

The model may generate incorrect information confidently.

Example:

Question

Who won the FIFA World Cup in 2030?

Without retrieval, the model may invent an answer.

2. Knowledge Cutoff

LLMs are trained on historical data.

They cannot automatically know:

- Today's news
 - Company documents
 - Personal files
 - Internal databases
-

3. Domain-Specific Knowledge

A hospital chatbot needs access to

- hospital policies
- doctor schedules
- treatment guidelines

This information is not part of the pretrained model.

4. Large Private Documents

Examples include:

- PDFs
- Books
- Contracts
- Research papers
- Annual reports

These documents exceed the context window.

Solution: Retrieval-Augmented Generation (RAG)

Definition

Retrieval-Augmented Generation (RAG) combines:

- Information Retrieval
- Vector Search
- Large Language Models

to generate accurate, grounded responses.

Instead of answering from memory alone, the chatbot first searches relevant documents.

High-Level Architecture

User

↓

Question

↓

Retriever

↓

Relevant Documents

↓

Prompt Construction

↓

Large Language Model

↓

Final Answer

The retrieval system acts like an "open-book exam."

Core Components of RAG

The pipeline contains five major stages:

1. Document Collection
2. Document Processing

3. Embedding Generation
 4. Retrieval
 5. Response Generation
-

Module 2: Document Collection

Sources include

- PDF
- Word
- Excel
- HTML
- Web pages
- Databases
- Emails
- Wikis
- APIs

Example:

University chatbot

Documents

- Admission brochure
 - Academic regulations
 - Hostel rules
 - Fee structure
 - Course syllabus
-

Module 3: Document Preprocessing

Raw documents must be cleaned.

Steps include

- Remove HTML tags
- Remove unnecessary spaces
- Normalize text
- OCR scanned PDFs
- Language detection
- Metadata extraction

Example

Original

Admission!!!
Please contact office...

Cleaned

Admission
Please contact office.

Module 4: Text Chunking

LLMs cannot process very large documents efficiently.

Therefore documents are divided into chunks.

Example

A 200-page PDF becomes

Chunk 1

Pages 1-3

Chunk 2

Pages 4–6

Chunk 3

Pages 7–9

Why Chunking?

Advantages

- Better retrieval
 - Lower memory
 - Faster search
 - Higher relevance
-

Chunking Strategies

Fixed-size chunking

500 words

Simple but may split ideas.

Sentence chunking

Break by sentences.

Better semantic preservation.

Paragraph chunking

Break by paragraphs.

Suitable for books.

Semantic chunking

Split when the topic changes.

Most advanced.

Sliding Window Chunking

Chunk 1

Sentence 1–10

Chunk 2

Sentence 8–18

Chunk 3

Sentence 16–26

Overlap preserves context.

Module 5: Embeddings

What is an Embedding?

An embedding is a numerical vector representing the semantic meaning of text.

Example

"I like AI"

↓

[0.21, -0.87, 0.63, ...]

Similar sentences have similar vectors.

Embedding Models

Popular embedding models include:

- Sentence Transformers
 - OpenAI Embeddings
 - BGE
 - E5
-

Embedding Workflow

Text

↓

Embedding Model

↓

Vector

↓

Vector Database

Module 6: Vector Database

A vector database stores embeddings.

Popular vector databases include:

- FAISS
- Pinecone
- Milvus
- Weaviate
- Chroma

Stored Information

Each record contains

Chunk Text

Embedding Vector

Metadata

Source

Page Number

Author

Module 7: Similarity Search

When a user asks

What is hostel fee?

The system

Question

↓

Embedding

↓

Vector Search

↓

Top Similar Chunks

Similarity Metrics

Cosine Similarity

Most popular.

Measures angle between vectors.

Range

-1 to 1

Closer to 1 means highly similar.

Euclidean Distance

Measures geometric distance.

Lower distance means higher similarity.

Dot Product

Used in high-dimensional vector search.

Approximate Nearest Neighbor (ANN)

Searching millions of vectors exactly is expensive.

ANN algorithms speed up search.

Popular methods

- HNSW
 - IVF
 - PQ
-

Module 8: Retriever

Retriever retrieves the most relevant chunks.

Input

Question

Output

Top-K Chunks

Example

Question

When is hostel admission?

Retriever returns

Chunk 14

Hostel admission starts on July 15.

Chunk 62

Students must submit ID.

Types of Retrieval

Dense Retrieval

Uses embeddings.

Semantic search.

Sparse Retrieval

Uses keywords.

Example

- BM25
 - TF-IDF
-

Hybrid Retrieval

Combines

- Dense
- Sparse

This usually improves recall.

Module 9: Re-ranking

Initial retrieval may return imperfect results.

A reranker sorts retrieved chunks by relevance.

Pipeline

Retriever

↓

Top 20

↓

Reranker

↓

Top 5

Popular rerankers include cross-encoder models from the Hugging Face ecosystem and other transformer-based architectures.

Module 10: Prompt Construction

Retrieved chunks are inserted into the prompt.

Example

System:

Answer only from context.

Context:

Hostel fee = ₹20,000.

Question:

Hostel fee?

Prompt Template

You are an assistant.

Use ONLY the supplied context.

If information is unavailable,

say

"I don't know."

Context

{retrieved documents}

Question

{user query}

Module 11: Large Language Model

The LLM receives

- User question
- Retrieved context
- Instructions

Then generates the final response.

Common choices include:

- GPT-4
 - Llama 3
 - Mistral
 - Gemma
-

Complete RAG Pipeline

Documents

↓

Cleaning

↓

Chunking

↓

Embedding



Vector Database



User Question



Embedding



Similarity Search



Top Chunks



Prompt



LLM



Answer

Module 12: Implementation Architecture

Typical Technology Stack

Layer	Technologies
Frontend	HTML, CSS, JavaScript, React
Backend	Python, FastAPI, Flask
LLM Framework	LangChain, LlamaIndex
Embeddings	Sentence Transformers, OpenAI
Vector Store	FAISS, Chroma, Pinecone
LLM	GPT, Llama, Mistral
Storage	PostgreSQL, MongoDB

Example Implementation Flow

Load PDFs

↓

Split Text

↓

Generate Embeddings

↓

Store in FAISS

↓

User Query

↓

Retrieve Top 5

↓

Pass to GPT

↓

Display Answer

Pseudocode

```
documents = load_documents()
```

```
chunks = split(documents)
```

```
embeddings = embed(chunks)
```

```
vector_db.store(chunks, embeddings)
```

```
query = user_input()
```

```
query_embedding = embed(query)
```

```
results =  
vector_db.search(query_embedding)
```

```
prompt = build_prompt(results, query)
```

```
answer = llm(prompt)
```

```
print(answer)
```

Module 13: Conversation Memory

Chatbots need memory for multi-turn conversations.

Types include:

- Buffer memory
 - Summary memory
 - Token-limited memory
 - Long-term memory using vector stores
-

Module 14: Advanced RAG Techniques

Query Expansion

Original:

AI

Expanded:

Artificial Intelligence

Machine Learning

Deep Learning

Improves recall.

Multi-Query Retrieval

Generate multiple reformulations of the same question and retrieve documents for each, improving coverage.

Parent-Child Retrieval

Retrieve a small chunk but return a larger surrounding section for richer context.

Knowledge Graph RAG

Combine vector search with graph databases to answer relationship-heavy questions.

Agentic RAG

An AI agent decides when to search, which tools to use, and how to synthesize results before responding.

Module 15: Evaluation Metrics

Evaluate a RAG chatbot using:

- **Retrieval Precision:** Fraction of retrieved chunks that are relevant.
- **Recall:** Fraction of all relevant chunks that were retrieved.
- **MRR (Mean Reciprocal Rank):** Rewards retrieving a correct result early.
- **nDCG:** Measures ranking quality while considering position.
- **Answer Faithfulness:** Whether the answer is supported by retrieved evidence.

- **Answer Relevance:** Whether the response addresses the user's question.
 - **Latency:** End-to-end response time.
 - **User Satisfaction:** Ratings, task completion, and feedback.
-

Advantages of RAG

- Reduces hallucinations
 - Uses up-to-date information
 - Supports private knowledge bases
 - Provides source-grounded responses
 - Cost-effective compared with frequent model retraining
 - Easier to update by replacing or adding documents
-

Limitations of RAG

- Retrieval errors can lead to incorrect answers.
 - Poor chunking reduces retrieval quality.
 - Embedding models may miss subtle meanings.
 - Large indexes increase storage and maintenance costs.
 - Prompt context windows still limit how much retrieved text can be included.
 - Requires careful evaluation and monitoring.
-

Applications

- Enterprise knowledge assistants
- Customer support
- Educational tutoring

- Legal document search
 - Healthcare information assistants
 - Research assistants
 - Financial advisory systems
 - Technical documentation search
 - Government information portals
-

Best Practices

1. Use high-quality, cleaned documents.
 2. Choose chunk sizes (typically 300–800 tokens) with overlap when appropriate.
 3. Store useful metadata (source, page, author, timestamp).
 4. Prefer hybrid retrieval for many real-world applications.
 5. Add reranking to improve precision.
 6. Instruct the LLM to answer only from retrieved context and acknowledge when information is unavailable.
 7. Include citations or links to source documents in responses where possible.
 8. Continuously evaluate retrieval and answer quality using benchmark questions and user feedback.
-

Summary

A **Retrieval-Augmented Generation (RAG)** chatbot enhances a large language model by retrieving relevant external information before generating a response. Its workflow consists of document ingestion, preprocessing, chunking, embedding generation, vector storage, semantic retrieval, optional reranking, prompt construction, and response generation. Compared with standalone LLMs, RAG systems

provide more accurate, explainable, and updatable answers, making them well suited for enterprise search, education, healthcare, customer support, and other knowledge-intensive applications.

Complete code step by step:

1. **Part 1:** Environment setup, project structure, install packages, and create sample documents.
 2. **Part 2:** Load PDFs/text files and preprocess them.
 3. **Part 3:** Split documents into chunks with explanations.
 4. **Part 4:** Generate embeddings using a sentence embedding model.
 5. **Part 5:** Create a FAISS vector database and save it.
 6. **Part 6:** Perform similarity search with examples and outputs.
 7. **Part 7:** Connect the retriever to an LLM (OpenAI or a local model).
 8. **Part 8:** Build a complete RAG chatbot with memory.
 9. **Part 9:** Build a web interface using Flask or Streamlit.
 10. **Part 10:** Explain every line of code, architecture diagrams, viva questions, interview questions, and troubleshooting.
-

Retrieval-Augmented Generation (RAG) Chatbot Tutorial

Part 1: Environment Setup and Project Structure

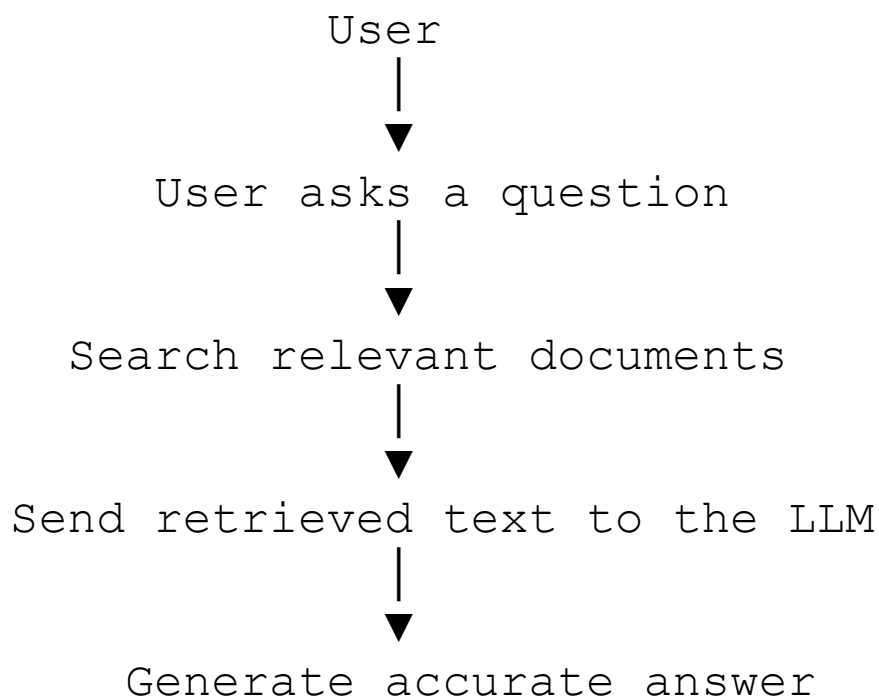
Learning Objectives

In this chapter, you will learn:

- What software is required
 - How to install the necessary libraries
 - How to create the project folder
 - What each library does
 - How the overall RAG project is organized
-

Step 1: What is a RAG Chatbot?

A RAG chatbot has two main components:



Unlike a normal chatbot, it **searches your documents before answering**.

Example:

You upload a university admission brochure.

User asks:

What is the admission fee?

The chatbot first searches the brochure, finds the relevant section, and then answers based on that text.

Step 2: Software Required

Install the following:

Software	Purpose
Python 3.10 or later	Programming language
VS Code	Code editor
Git (optional)	Version control
Terminal/Command Prompt	Run commands

Verify Python Installation

Open Terminal or Command Prompt.

Type:

```
python --version
```

Expected Output

```
Python 3.11.5
```

Step 3: Create Project Folder

Create a new folder.

RAG_Chatbot/

Inside it create

```
RAG_Chatbot/  
├── data/  
├── vector_db/  
├── app.py  
├── requirements.txt  
└── README.md
```

Explanation

data/

Stores documents.

Example

Admission.pdf

Rules.pdf

Fees.pdf

vector_db/

Stores vector embeddings.

Initially empty.

Later it becomes

```
vector_db/
```

```
index.faiss
```

```
index.pkl
```

app.py

Main chatbot program.

requirements.txt

Contains all required libraries.

Step 4: Create Virtual Environment

A virtual environment keeps project dependencies isolated.

Windows

```
python -m venv venv
```

Linux/Mac

```
python3 -m venv venv
```

Activate it.

Windows

```
venv\Scripts\activate
```

Linux

```
source venv/bin/activate
```

Expected output

```
(venv)
```

The `(venv)` prefix indicates the environment is active.

Step 5: Install Required Libraries

Run:

```
pip install langchain
```

Next:

```
pip install langchain-community
```

Next:

```
pip install faiss-cpu
```

Next:

```
pip install sentence-transformers
```

Next:

```
pip install pypdf
```

Finally:

```
pip install openai
```

Install All Together

Instead of one by one, you can run:

```
pip install langchain langchain-community  
sentence-transformers faiss-cpu pypdf  
openai
```

Step 6: Save Dependencies

Run:

```
pip freeze > requirements.txt
```

Example requirements.txt:

```
langchain==0.3.x  
langchain-community==0.3.x  
sentence-transformers==3.x  
faiss-cpu==1.x  
pypdf==5.x  
openai==1.x
```

This file allows others (or future you) to recreate the environment with:

```
pip install -r requirements.txt
```

Step 7: Create Sample Document

Create a file:

```
data/university.txt
```

Paste:

ABC University Admission Guide

Admission opens on July 15.

The admission fee is Rs. 500.

Hostel fee is Rs. 20,000 per semester.

Library timing is 9 AM to 8 PM.

Students must carry their ID card.

Explanation

This is the **knowledge base**.

The chatbot will search this file when answering questions.

Step 8: Test Python Installation

Create

`app.py`

Write:

```
print("Welcome to RAG Chatbot")
```

Run:

```
python app.py
```

Output

Welcome to RAG Chatbot

Step 9: Import Required Libraries

Update `app.py`:

```
from langchain_community.document_loaders
import TextLoader
```

```
print("Libraries imported successfully!")
```

Run:

```
python app.py
```

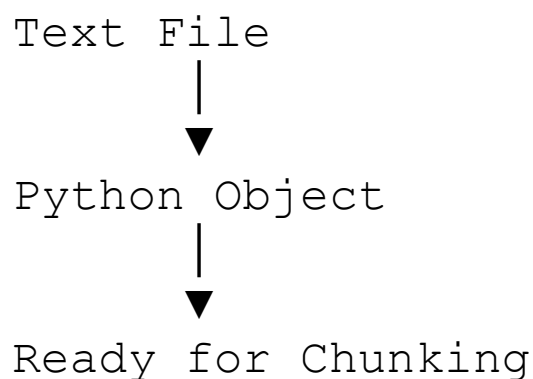
Output

```
Libraries imported successfully!
```

What Does `TextLoader` Do?

It loads a text document into a format that LangChain can process.

Think of it as:



Step 10: Load the Document

Replace `app.py` with:

```
from langchain_community.document_loaders
import TextLoader

loader = TextLoader("data/university.txt")

documents = loader.load()

print(documents)
```

What Happens Internally?

`university.txt`

↓

`TextLoader`

↓

`Document Object`

LangChain wraps the file content in a `Document` object, which stores both the text and metadata (such as the source file).

Expected Output

```
[Document (
page_content='ABC University Admission
Guide
```

Admission opens on July 15.

The admission fee is Rs. 500.

Hostel fee is Rs. 20,000 per semester.

Library timing is 9 AM to 8 PM.

```
Students must carry their ID card.',  
metadata={'source': 'data/university.txt'}  
)]
```

Understanding the Output

The Document object contains:

- `page_content`: the actual text.
- `metadata`: information about the source.

This metadata helps later when we want to cite where an answer came from.

Summary of Part 1

You have:

- ✓ Installed Python
- ✓ Created a virtual environment
- ✓ Installed the required libraries
- ✓ Created the project structure
- ✓ Added a sample knowledge base
- ✓ Loaded the document using LangChain

What's Next? (Part 2)

In **Part 2**, we'll cover:

- Why we **split documents into chunks**
- How to use `RecursiveCharacterTextSplitter`
- Choosing chunk size and overlap
- Viewing the generated chunks
- Complete code with detailed explanations and expected output
- Why chunking is critical for retrieval accuracy

By the end of Part 2, you'll understand how a long document is transformed into searchable pieces before generating embeddings.